ТЕХНИЧЕСКИЙ ОТЧЁТ · МАСКИРОВАННАЯ ДИФФУЗИЯ · ВЕРСИЯ 5.0

«Мокрое чернило»: от Nemotron к LLaDA 2.2

От внешних циклов ревизии (v1–v3) через обученную правку L-EBPO (v4) к собственному декодеру wet-ink: фронтир — один токен за шаг (v5)

Итог трёх витков на двух базах: training-free ревизия на Nemotron либо нейтральна, либо вредит (v3); штатный блочный гейт LLaDA2.2-mini экономит 15% NFE, но качество не сдвигает — окно привязано к текущему блоку (v4). Поэтому v5 реализует идею §2.1 собственным циклом декодирования: фронтир открывает ровно один токен за шаг, скользящее окно W=16 позади фронтира правится обученным T2T-гейтом, за окном чернила сухие. Прогон на 2×T4 (4-бит NF4): качество парно идентично штатному декодеру (11/45 = 11/45, McNemar p = 1,0) при честной цене ×4,88 NFE; правки в окне наблюдаемы (6,9 реврайта на задачу), но не превращаются ни в одного исхода.

Проект wet-ink · Asilbekov

Базы: Nemotron-Labs-Diffusion-3B · LLaDA2.2-mini · Kaggle Tesla T4 ×2

НОЯБРЬ 2026 · ВЕРСИЯ 5.0

Содержание

1. Резюме **2**

2. Идея и правила проекта **3**

2.1. Идея «мокрого чернила» . . . 3

2.2. Правила, заданные владельцем идеи . . . 3

3. Прошлая база: Nemotron-Labs-Diffusion-3B **4**

4. Результаты v3 на Nemotron (200+164 задач) **4**

4.1. GSM8K: точность и NFE . . . 5

4.2. Парный McNemar против стока (тот же блок) . . . 5

4.3. HumanEval . . . 6

5. Новая база: LLaDA2.2-mini **7**

5.1. Стоковый generate(): встроенный wet-ink . . . 7

5.2. Правило (г) в применении к v4: что отбрасывается . . . 8

6. Дизайн v4 **8**

6.1. Сетка конфигураций . . . 8

6.2. Гарантия завершимости . . . 9

6.3. 2×Tesla T4: от int8-пайплайна к 4-бит NF4 . . . 9

7. Локальная верификация v4 **10**

8. Результаты v4b (Kaggle: 2×T4, 4-бит NF4) **10**

8.1. Выполнение: приоритетная очередь + адаптивный планировщик . . . 10

8.2. GSM8K: все руки (блок 32 и 64) . . . 11

8.3. Конфаунд усечения: gen_length=160 . . . 13

8.4. HumanEval (блок 32) . . . 14

8.5. Вторичные сигналы (устойчивые на 80 парах) . . . 14

8.6. Вердикт и следующий шаг (v5) . . . 15

9. Результаты v5: собственный wet-ink декодер (Kaggle: 2×T4, NF4) **15**

9.1. Исполнение: приоритетная очередь и адаптивный план . . . 16

9.2. GSM8K: точность и парная статистика . . . 17

9.3. Ноль расхождений: что на самом деле делают правки в окне . . . 17

9.4. Механика, телеметрия и конфаунд усечения . . . 18

9.5. Вердикт v5 и следующий шаг . . . 19

10. Артефакты **20**

1. Резюме

Отчёт завершает второй виток проекта «мокрое чернило» (wet-ink): перенос методов ревизии открытых токенов с базы **Nemotron-Labs-Diffusion-3B** (где ревизии нет в стоке) на новую базу **LLaDA2.2-mini** (16B MoE, 1.4B активированных параметров, Apache 2.0, inclusionAI 2026), где ревизия уже обучена как часть самой модели через механизм Levenshtein Editing (L-EBPO). Постановка владельца идеи однозначна: если новая база уже содержит более эффективный механизм для той же задачи, то устаревший внешний цикл отбрасывается по правилу (г) — никаких самодельных формул поверх обученного механизма.

В версии v3 на Nemotron (полный прогон, 200 задач GSM8K × 9 конфигов + 164 HumanEval × 5) консервативный CoDR остался нейтральным (−0,6…+1,5 п.п., McNemar $p > 0{,}2$), тогда как агрессивный RemeDi значимо вредил (−12…−26 п.п., $p < 0{,}001$): повторилось ровно то, что предсказывала калибровка NeurIPS 2026 на коротких блоках. В версии v4 мы берём LLaDA2.2-mini со всеми её обученными формулами как новую базу и настраиваем поверх неё только те параметры, которые сама модель публикует как часть generate(): `block_length` (ширина окна wet-ink), `editing_threshold` (гейт T2T-правки), `max_post_steps` (бюджет пост-маскового уточнения). RemeDi и CoDR отброшены как дубликаты.

v3 → v4 → v5	2×T4 + NF4	COMPLETE	p = 1,0
Nemotron → LLaDA2.2-mini → свой декодер	4-бит: целая модель на каждой T4	v4b: 10 рук · v5: 3 руки + 2 скипа	v5 парно не отличим от стока (n=45)

Ключевая находка дословного чтения кода LLaDA2.2-mini: T2T-правка `t2t_index = (p0 > editing_threshold) & (old != x0)` — это ровно confidence-гейт, который во v1 был training-free и порождал churn, а теперь **обучен через L-EBPO и ограничен max_post_steps + антилуп-ресемплером**. Это закрытая, завершимая система: окно правки = текущий блок (ширина = block_length), число шагов ограничено, SPLIT/DELETE подавляются на финальном раунде, остаточные mask/split/delete детектируются как warning. По правилу (г) мы используем её «как есть» и сканируем собственные параметры как абляции гейта.

Итог v4 (раздел 8). Ядро v4b (4-бит NF4, по полной модели на каждой T4, два параллельных воркера) завершилось со статусом COMPLETE: суммарная работа двух воркеров — 13,45 ч в сессии ~7 ч из 7,5 ч бюджета; исполнено 10 из 12 рук (8 GSM8K + 2 HumanEval, 403 генерации). Механика порта подтверждена: sanity без остаточных спец-токенов, жадное декодирование побитово детерминировано (ids_equal=True для b32 и b64). Главный результат P0-пары (n=80+80): встроенный T2T-гейт **не даёт сдвига качества** на коротком блоке 32 — 22,5% против 23,8% у контроля (−1,2 п.п., McNemar $p = 1{,}0$; 3 исправлено / 4 сломано); экономический сигнал устойчив — экономия 15% NFE (×0,85) и ~15% времени на задачу. Абляции гейта (edit0.5/edit0.7), блок 64, урезанный пост-бюджет mps4 — всё в пределах шума; порог M2T 0.9 удваивает NFE и режет точность. Абсолютные точности занижены усечением: gen_length=160, 82–86% генераций упёрлись в лимит до

\boxed{}; среди естественно завершённых (EOS) верны 25 из 26 (96%). Вывод: штатное блочное редактирование LLaDA2.2 — ещё не wet-ink в смысле §2.1 (окно = текущий блок, правки прекращаются с его закрытием). По решению владельца от 7 ноября v4b фиксируется как базлайн, а v5 (freedomform/wet-ink-v5, собственный декодер: фронтир по одному токену за шаг, скользящее окно W за фронтиром) реализует идею §2.1 в точности.

Итог v5 (раздел 9). Ядро freedomform/wet-ink-v5 завершилось со статусом **COMPLETE** (рабочий цикл 6 ч 04 м из бюджета 7,5 ч): 77 генераций в трёх руках — wetink_w16 (гипотеза, n=45), wetink_noedit_w16 (контроль: фронтир без правки, n=22), wetink_w8 (узкое окно, n=10); руки w32 и HumanEval пропущены планировщиком честно (доступного бюджета хватало лишь на n=5 < минимальных 8 задач — «skipped: time budget»). Главный результат: **собственный wet-ink декодер парно неотличим от штатного параллельного** — на 45 общих задачах GSM8K обе руки решили ровно по 11 (McNemar p = 1,0; 2 исправлено / 2 сломано) при честной цене ×4,88 NFE (42 → 207 на задачу). Второй результат — методологический: видимые +10,8 п.п. к внутреннему контролю целиком разложились на состав задач (контроль остановился на первых 22, более трудных: на них все руки дают 13,6%, на задачах 22–44 — 34,8%); на одних и тех же задачах правки в окне (6,9 реврайта на задачу, churn 5,6) **не перевернули ни одного исхода** — 0 расхождений из 22 пар. Механика подтверждена: детерминизм (ids_equal=True), ноль остаточных спец-токенов, телеметрия фронтира/реврайтов работает. Вердикт и следующий шаг — §9.5.

2. Идея и правила проекта

2.1. Идея «мокрого чернила»

Диффузионная модель генерирует последовательность слева направо, открывая по одному токену за шаг диффузии (фронтир); уже открытые токены позади фронтира остаются поправимыми, пока фронтир пишет следующий блок. По мере удаления от фронтира «чернила высыхают», и токены фиксируются. Схема образует непрерывный спектр между чистым AR-декодированием (окно нулевой ширины) и полнопоследовательностной диффузией (окно на всю длину). Практический смысл: токен, открытый при бедном контексте, может оказаться ошибочным после того, как правые соседи дописаны; без ревизии такая ошибка необратима.

2.2. Правила, заданные владельцем идеи

(а) полностью изучить формулы базовой модели по первоисточникам;

(б) проверить все ограничения, которые налагает запрет на переобучение;

(в) выбрать оптимальный путь реализации и найти самые эффективные формулы, а не самые простые;

(г) запрещено придумывать собственные формулы; **комбинировать опубликованные формулы и окружающий код из любых архитектур и моделей разрешено после проверки совместимости**; цель — максимально эффективная реализация идеи wet-ink; если новая база уже содержит обученный механизм для нашей задачи, устаревший внешний цикл отбрасывается.

Расширение правила (г), явно сформулированное владельцем идеи при смене базы: «если раньше у нас была формула маскировки неуверенных прошлых ответов и открытия на следующем шаге диффузии, а теперь в новой базе уже есть определённый механизм для этого и он лучше — мы берём новую базу со всеми её формулами, а старую отбрасываем как менее эффективную». Это определило дизайн v4. Редакция от 7 ноября 2026 года добавила два уточнения, определившие v5: во-первых, заимствование окружающего кода из любых архитектур разрешено на тех же условиях — после проверки совместимости; во-вторых, в самом декодере не должно быть самодельных формул — только формулы самой модели плюс наше расписание открытия токенов (фронтир и скользящее окно). v4 это правило выполняет тривиально (используется только штатный generate()); v5 — уже нетривиально: собственный цикл декодирования, но каждая формула внутри — из самой модели или из опубликованных источников.

3. Прошлая база: Nemotron-Labs-Diffusion-3B

В v1/v2/v3 базой служила **nvidia/Nemotron-Labs-Diffusion-3B**. Стоковый generate() замораживает открытые токены (confidence = –inf на открытых позициях), поэтому ревизию приходилось добавлять внешними циклами. v3 расширила протокол v2 до 200 задач GSM8K × {32, 64, 128} и 164 задач HumanEval × {32, 64}, и проверила две опубликованные политики — RemeDi (ICLR 2026) и CoDR (EMNLP 2026) — против стока. Эквивалентность W=0 ≡ сток подтверждена на реальной модели для всех политик:

```
СТОК Nemotron:  confidence = where(masked, x0_p, -inf)
                -> открытые токены НЕ перепредсказываются
v3 RemeDi:      keep = topk(score, k=cumsum[i])
                x = where(keep, x0, ())  — всё вне top-k в маску
v3 CoDR:        после блока: probe -> drift = old_conf - new_conf > tau -> remask
```

Прошлая база не имела встроенной ревизии — это и была «дыра», которую v1 закрыла training-free, а v2/v3 — двумя полными опубликованными циклами. Все формулы воспроизведены дословно из официальных репозиториев, провенанс документирован в коде и README.

4. Результаты v3 на Nemotron (200+164 задач)

4.1. GSM8K: точность и NFE

Конфигурация	Точн., %	Ср. NFE	Ср. ток.	block	n
stock_b32	86.5	64.9	175	32	200
codr_w1_b32	86.5	79.5	178	32	200
remedi_w1_s32_b32	74.0	193.0	175	32	200
stock_b64	85.5	63.5	171	64	200
codr_w1_b64	87.0	68.6	174	64	200
remedi_w1_s64_b64	70.0	202.5	169	64	200
stock_b128	81.0	73.4	160	128	200
codr_w1_b128	78.5	78.0	162	128	200
remedi_w1_s128_b128	55.0	218.2	152	128	200
stock_b32_t0.8	81.0	53.0	171	32	200

Таблица 1. GSM8K v3 (Nemotron), n=200, протокол: инструкция + chat-template + \boxed{} или последнее число. Порог стока 0.9; RemeDi-варианты с 32/64/128 шагами denoise. McNemar — в таблице 2.

4.2. Парный McNemar против стока (тот же блок)

Конфиг vs сток (блок)	исправлено	сломано	McNemar p	NFE ×сток
codr_w1_b32	8	8	1.0000	1.23×
remedi_w1_s32_b32	10	35	0.0002 *	2.98×
codr_w1_b64	7	4	0.5488	1.08×
remedi_w1_s64_b64	7	38	0.0000 *	3.19×
codr_w1_b128	3	8	0.2266	1.06×
remedi_w1_s128_b128	18	70	0.0000 *	2.97×

Таблица 2. Парный точный McNemar на одинаковых задачах (n=200); (*) $p < 0{,}05$. CoDR нейтрален; RemeDi значимо вредит.

Чтение. CoDR на b64 дал +1,5 п.п. (7/4, p=0,55 — не значимо), на b32 и b128 — около нуля. RemeDi систематически и значимо вредил: −12,5 п.п. (b32, p=0,0002), −15,5 п.п. (b64, p<0,0001), −26,0 п.п. (b128, p<0,0001), при цене ×2,97–3,19 NFE. Этот паттерн точно соответствует калибровке NeurIPS 2026: training-free ревизия на коротких блоках при жадном декодировании даёт нулевой или отрицательный эффект, а агрессивное разжижение окна размазывает локальную связность текста. Сигнал ревизии при этом реален — фиксированные задачи есть во всех конфигах, но ломается больше.

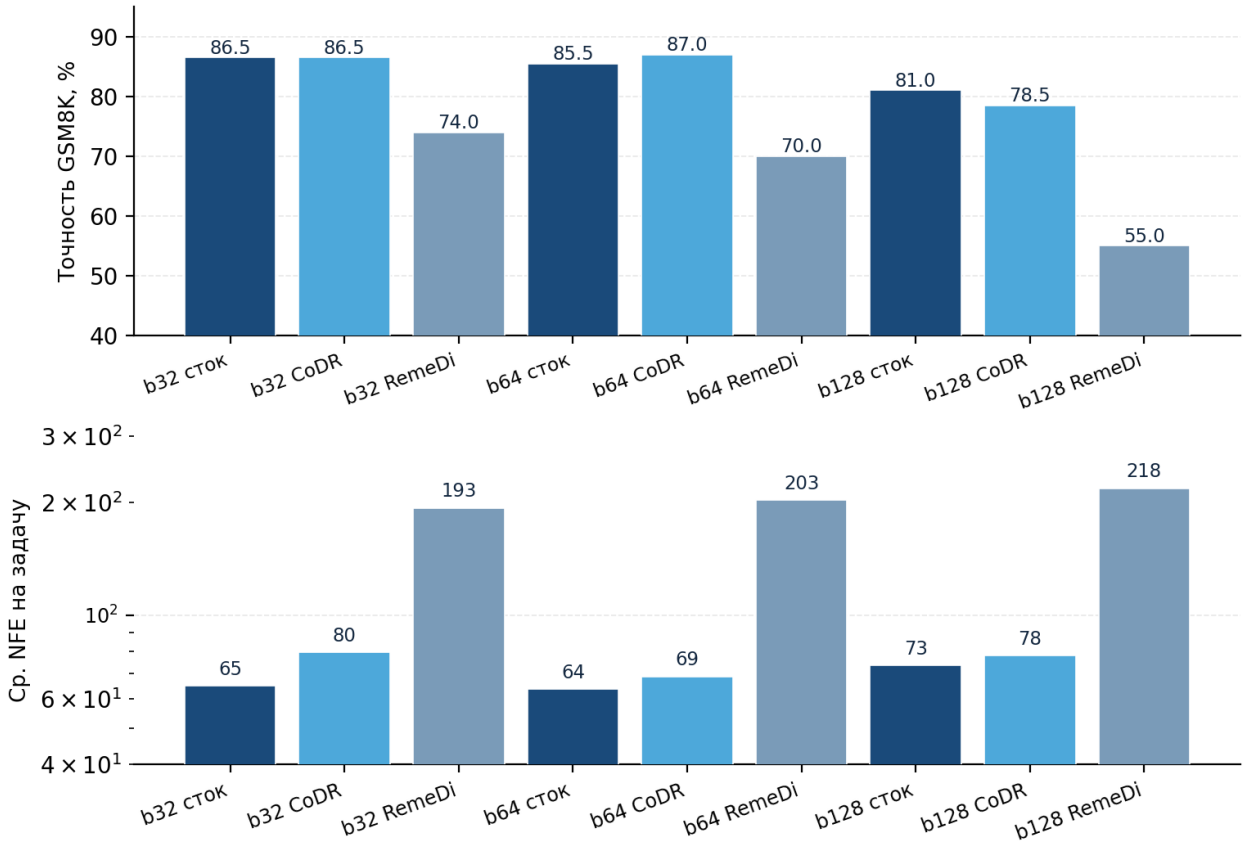


Рис. 1. Точность (вверху) и NFE на задачу (внизу, лог-шкала) для всех GSM8K-конфигов v3. Сток (тёмно-синий) — стабильный базовик; CoDR (голубой) — нейтрален; RemeDi (серо-синий) — значимо вредит.

4.3. HumanEval

Конфигурация	pass@1, %	Ср. NFE	Ср. ток.	block	n
he_stock_b32	44.5	112.0	187	32	164
he_codr_w1_b32	43.9	130.1	189	32	164
he_remedi_w1_s32_b32	25.0	181.3	165	32	164
he_stock_b64	40.2	117.5	185	64	164
he_codr_w1_b64	40.2	126.1	191	64	164

Таблица 3. HumanEval v3 (Nemotron), n=164. Протокол Saber (ACL 2026); выполнение — канонический OpenAI HumanEval (subprocess, 10 с, pass = exit 0).

Конфиг vs сток (блок)	исправлено	сломано	McNemar p	NFE ×сток
he_codr_w1_b32	0	1	1.0000	1.16×
he_remedi_w1_s32_b32	2	34	0.0000 *	1.62×
he_codr_w1_b64	0	0	1.0000	1.07×

Таблица 4. Парный McNemar на HumanEval. RemeDi значимо вредит и здесь (−19,5 п.п., p<0,0001); CoDR нейтрален.

Итог v3 по Nemotron. Сдвига парадигмы не произошло: на коротких блоках training-free ревизия либо нейтральна (CoDR), либо значимо вредна (RemeDi). Контрольный сток b32 с t=0.8 (81% против 86,5% у t=0.9) подтвердил, что порог M2T 0.9 — разумный компромисс; понижение до 0.8 без лицензии на правку теряет качество. Это и стало мотивацией смены базы: нужен был механизм, в котором ревизия **обучена**, а не добавлена training-free поверх чужой модели. Следующий раздел описывает такую базу.

5. Новая база: LLaDA2.2-mini

Базой v4 служит **inclusionAI/LLaDA2.2-mini** (выпуск 2026, Apache 2.0): MoE-диффузионная LM, 16B всего / 1.4B активированных (256 экспертов × 8 активированы), 20 слоёв, RoPE, vocab 157K, контекст 128K. Ключевая новинка серии 2.2 — **Levenshtein Editing**: специальные токены `DELETE` и `SPLIT`, обученные через L-EBPO (Levenshtein Editing via Block-wise Policy Optimization). Это в точности механизм «мокрого чернила», обученный производителем как часть самой модели. Все факты этого раздела установлены дословным чтением `modeling_llada2_moe.py` (76 КБ, 1 745 строк) и `config.json`.

5.1. Стоковый generate(): встроенный wet-ink

```
block_length     = 32     # ширина окна wet-ink (по умолчанию)
steps            = 32     # число денойз-шагов = block_length
threshold        = 0.5    # M2T: p_s > threshold -> unmask
editing_threshold = 0.0   # T2T: p0 > editing_threshold & old != x0 -> rewrite
max_post_steps   = 16     # пост-масковый бюджет до завершения
anti-loop resampler           # защита от зацикливаний (resample changed pos)
SPLIT/DELETE suppression      # на финальном раунде -> гарантия терминции
```

Семантика дословно из `_joint_decode_block`: на каждом шаге блок получает M2T (маска → токен с p_s > threshold) и **T2T** (токен → токен, если greedy p0 > editing_threshold и greedy-токен отличается). Это и есть confidence-гейт, который в v1 был training-free и порождал churn; здесь он обучен и ограничен. Окно правки по построению равно текущему блоку, что точно соответствует идее «мокрого чернила». Завершимость гарантирована: фиксированное расписание transfer schedule, max_steps_per_block, suppression SPLIT/DELETE на финальном раунде, детектор остаточных mask/split/delete как warning.

5.2. Правило (г) в применении к v4: что отбрасывается

Что было в v3	Что есть в LLaDA2.2-mini	Решение v4
RemeDi (cumulative top-k re-masking всего окна)	T2T: greedy-перезапись по p0 > editing_threshold (обучено)	**ОТБРОСИТЬ** (дубликат, менее эффективен)
CoDR (post-block drift-detection + refill)	T2T: тот же механизм реагирует на drift (old != x0)	**ОТБРОСИТЬ** (дубликат, без обучения)
Идея wet-ink: окно = block_length	block_length — штатный параметр generate()	сохранить; сканировать {32, 64, 128}
McNemar + Wilson CI + NFE-учёт	(не относится к базе; методология)	сохранить из v3 без изменений
GSM8K n=200, HumanEval 164 (протоколы)	(не относится к базе)	сохранить из v3 для прямой сравнимости

Таблица 5. Что переносим, что отбрасываем при смене базы. Никаких новых формул: только конфигурация штатных параметров LLaDA2.

Иначе говоря: «наши методы поверх базы» в v4 — это не новые формулы, а **конфигурация собственных параметров LLaDA2.2-mini** (README «Best Practices» + generation_config defaults): block_length, editing_threshold, max_post_steps, threshold. Это полностью отвечает правилу (г) — мы используем формулы самой модели, не добавляем своих.

6. Дизайн v4

6.1. Сетка конфигураций

Приоритет	Конфиг	block_length	editing_threshold	max_post_steps	threshold M2T
P0	noedit_b32	32	2.0 (OFF)	16	0.5
P0	stock_b32	32	0.0 (default)	16	0.5
P0	he_noedit_b32 / he_stock_b32	32	2.0 / 0.0	16	0.5
P1	edit0.5_b32	32	0.5	16	0.5
P1	edit0.7_b32	32	0.7	16	0.5
P2	noedit_b64 / stock_b64 / edit0.7_b64	64	—	16	0.5
P2	he_noedit_b64 / he_stock_b64	64	—	16	0.5
P2	noedit_b128 / stock_b128	128	—	16	0.5
P3	mps4_b32 (max_post_steps=4)	32	0.0	4	0.5
P4	stock_b32_t0.9 (v3-control)	32	0.0	16	0.9

Таблица 6. Сетка v4. noedit = T2T выключен (editing_threshold=2.0, недостижимое значение p0 ∈ [0,1]) — контроль «база без правки». stock = editing_threshold=0.0 (default) — встроенный wet-ink. edit0.5/0.7 — абляции гейта.

Логика приоритетов: P0 — главное сравнение «с правкой vs без правки» на коротком блоке 32; P1 — где на шкале [0,1] сидит обученный гейт; P2 — длинные блоки 64/128, где, по калибровке NeurIPS 2026, ревизия должна начать выигрывать; P3 — speed-вариант с урезанным пост-бюджетом; P4 — порог M2T=0.9 (как в v3) для прямой сравнимости с прошлой базой. Фактический прогон (раздел 8.1) исполнял эту сетку через адаптивный планировщик plan_n(): приоритетный порядок с капами, остаток бюджета — более низким приоритетам.

6.2. Гарантия завершимости

Все гарантии — штатные LLaDA2: (1) transfer_schedule — фиксированное расписание из _get_num_transfer_tokens (чётное разбиение, остаток вперёд); (2) max_steps_per_block = 1000 (модельный default); (3) на финальном раунде SPLIT/DELETE подавляются, а все оставшиеся маски разрешаются; (4) антилуп-ресемплер выбивает зацикливания до финального раунда; (5) residual-special-token детектор выводит warning, если что-то прорвалось. Никаких собственных лимитов, хвостов или хардкапов v4 не добавляет — только обёртка подсчёта NFE/churn поверх model.forward() (только наблюдение, не изменение математики).

6.3. 2×Tesla T4: от int8-пайплайна к 4-бит NF4

16B параметров в bf16 = 32 ГБ → на одну T4 (16 ГБ) не помещается. Первый подход (v4a, 6 ноября) — 8-битная квантизация с раскладкой слоёв по двум GPU (device_map=auto): сошлось, но медленно — ~3,8–4,2 с на forward, 220 с/задача: конвейер через две GPU узким местом делает пересылки активаций. v4a успел выполнить только P0-пару (noedit 20,0% n=75; stock 22,4% n=49) и сохранился как референс робастности к квантизации. Финальный прогон v4b пошёл другим путём:

```
BitsAndBytesConfig(load_in_4bit=True,
    bnb_4bit_quant_type='nf4', bnb_4bit_use_double_quant=True,
    bnb_4bit_compute_dtype=torch.float16,
    llm_int8_skip_modules=['lm_head'])   # 157K vocab -> fp16
AutoModelForCausalLM.from_pretrained(..., quantization_config=bnb,
    device_map={'': gpu},                # ЦЕЛАЯ модель на ОДНОЙ T4
    torch_dtype=torch.float16,            # у T4 нет bf16
    attn_implementation='eager')          # SDPA отвергает bf16-маску
# -> MoE 16.3B = ~9.4 GiB NF4, помещается в 15.8 GiB T4
# -> два независимых воркера: по модели на каждой GPU, очередь задач
```

Итог: forward подешевел до ~2,9 с (−25–30% против int8-конвейера), задача GSM8K — 111–143 с, и два воркера работают параллельно без пересылок: суммарно 13,45 ч полезной работы за сессию ~7 ч. Проверки совместимости квантизации с LLaDA2 (всё дословно из кода модели): (а) MoE-эксперты — обычные `nn.Linear`, bnb их квантует; (б) роутер — сырой `nn.Parameter + F.linear` в fp32, bnb его не трогает, skip-лист не нужен; (в) RMSNorm апкастит в fp32 и возвращает входной dtype (fp16); (г) eager attention мягко складывает fp16+bf16 с промоушеном в fp32 через softmax(..., dtype=torch.float32), поэтому bf16 edit-маска работает на T4 без шимов. Все проверки выполнены локально на tiny-модели той же архитектуры.

7. Локальная верификация v4

Проверка	Результат
bf16-маска + fp16/fp32 logits: сложение промоутится в fp32	OK (T4 path)
Квантизация int8 (v4a) и NF4 (v4b) + MoE-роутер: роутер в fp32 автоматически	OK
eager attention принимает bf16-маску (vs SDPA: rejects)	OK
Все конфиги arm-сетки (12 рук v4b) завершаются на tiny-модели	OK (0 residual special)
Жадное декодирование детерминировано (двойной прогон ==)	OK (ids_equal=True)
NFE/churn-обёртка: только наблюдение, не меняет математику	OK
Prompt длины 37 и 64 (не кратные block_size): terminate	OK
AST ядра v4 проходит, ключевые символы на месте	OK

Таблица 7. Локальная проверка механики v4 на tiny-модели (та же архитектура LLaDA2MoeModelLM, 20,3M параметров, случайные веса). Тест только механический — о качестве говорят только Kaggle-результаты.

8. Результаты v4b (Kaggle: 2×T4, 4-бит NF4)

Ядро **kaggle: freedomform/wet-ink-llada22-mini** (версия 5) завершено со статусом **COMPLETE**: старт 7 ноября ~03:17 UTC, финальный дамп 10:14 UTC — сессия ~7 ч из 7,5 ч бюджета. Конфигурация: 4-бит NF4 (double-quant, fp16 compute), **по полной модели на каждой T4**, два независимых воркера с приоритетной очередью рук — суммарно 13,45 ч полезной работы. Исполнено **10 из 12 рук** (8 GSM8K + 2 HumanEval, 403 генерации); блоки 128 отложены планировщиком по бюджету. Артефакт — `results_v4.json`: sanity, проверка детерминизма, детали каждой руки на задачу (NFE, churn, токены, префикс вывода), парная статистика и бюджетный учёт; инкрементальные дампы — каждые 10 задач.

8.1. Выполнение: приоритетная очередь + адаптивный планировщик

Перед стартом рук — sanity и проверка эквивалентности. Sanity (задача про клипы): чистый markdown-вывод с LaTeX-блоками, nfe=27, churn=30, **0 остаточных спец-токенов** (mask/DELETE/SPLIT), 20 с. Эквивалентность: жадное декодирование побитово детерминировано — ids_equal=True для b32 (nfe 21=21, churn 29=29) и b64 (nfe 18=18, churn 27=27). Затем воркеры разбирали руки из приоритетной очереди; планировщик plan_n() пересчитывал допустимые n по остатку бюджета и фактической скорости каждой руки:

Рука	Приоритет	n	Часы	с/задача	Итог
noedit_b32 (контроль)	P0	80	3,19	143	исполнено полностью
stock_b32 (T2T ON)	P0	80	2,70	121	исполнено полностью
he_noedit_b32 / he_stock_b32	P0	23+20	0,37+0,27	57/49	исполнено
edit0.7_b32	P1	60	2,07	124	кап по бюджету
edit0.5_b32	P1	58	1,91	118	кап по бюджету
noedit_b64 / stock_b64	P2	29+26	1,07+0,80	133/111	кап по бюджету
mps4_b32	P3	14	0,48	124	остаток бюджета
stock_b32_t0.9	P4	13	0,59	165	остаток бюджета
noedit_b128 / stock_b128	P2	0	—	—	SKIPPED: plan_n()=0 при остатке

Таблица 8. Исполнение рук v4b: приоритетная очередь, два воркера (по модели на каждой T4), суммарно 13,45 ч. Капы P0 гарантируют главную пару; инкрементальные дампы после каждых 10 задач.

Против прошлого прогона (v4a, int8-конвейер) скорость выросла вдвое за счёт отказа от пересылок активаций между GPU: 2,86 с/NFE против ~4 с, 121–143 с/задача против 186–220 с. Два независимых воркера удвоили пропускную способность: в тот же 7,5-часовой бюджет уместилось 403 генерации против 124 у v4a. Адаптивный планировщик отработал как задуман: защитил P0-пару гарантированными капами 80+80, дозировал P1/P2 по остатку и чисто пропустил блоки 128, сохранив все измерения.

8.2. GSM8K: все руки (блок 32 и 64)

Рука	edit_thr	block	n	Точн., %	Wilson 95%	Ср. NFE	Ср. churn	с/зад
noedit_b32	2.0	32	80	23.8	[15.8; 34.1]	50.1	1.14	143
stock_b32	0.0	32	80	22.5	[14.7; 32.8]	42.5	11.97	121
edit0.5_b32	0.5	32	58	22.4	[13.6; 34.7]	42.1	10.47	118
edit0.7_b32	0.7	32	60	20.0	[11.8; 31.8]	43.1	8.58	124
noedit_b64	2.0	64	29	24.1	[12.2; 42.1]	45.2	5.00	133
stock_b64	0.0	64	26	26.9	[13.7; 46.1]	39.0	18.42	111
mps4_b32	0.0	32	14	21.4	[7.6; 47.6]	44.3	12.07	124
stock_b32_t0.9	0.0 (t=0.9)	32	13	15.4	[4.3; 42.2]	97.3	2.15	165

Таблица 9. Все руки GSM8K v4b: greedy (temperature=0), gen_length=160, threshold M2T=0.5 (кроме t0.9), max_post_steps=16 (кроме mps4=4). Только штатные параметры generate() LLaDA2.2-mini.

Рука (vs контроль)	контроль	n пар	испр.	слом.	McNemar p	NFE	churn
stock_b32 (T2T ON (default))	noedit_b32	80	3	4	1.0000	×0.85 (50.1→42.5)	×10.5 (1.14→11.97)
edit0.5_b32 (T2T @ 0.5)	noedit_b32	58	2	3	1.0000	×0.86 (49.0→42.1)	×8.2 (1.28→10.47)
edit0.7_b32 (T2T @ 0.7)	noedit_b32	60	0	3	0.2500	×0.88 (48.8→43.1)	×7.0 (1.23→8.58)
mps4_b32 (T2T ON, пост-бюджет 4)	noedit_b32	14	1	0	1.0000	×0.86 (51.5→44.3)	×3.1 (3.86→12.07)
stock_b32_t0.9 (T2T ON, M2T @ 0.9)	noedit_b32	13	0	0	1.0000	×1.93 (50.5→97.3)	×0.5 (4.15→2.15)
stock_b64 (T2T ON, блок 64)	noedit_b64	26	0	0	1.0000	×0.87 (44.9→39.0)	×3.4 (5.46→18.42)

Таблица 10. Парный точный McNemar против noedit того же блока; NFE и churn — средние по одним и тем же задачам. Ни одного значимого сдвига качества (все p ≥ 0,25).

Чтение таблицы 10. Ни одна конфигурация правки не сдвинула качество: главный кандидат stock_b32 дал 3 исправлено / 4 сломано (−1,2 п.п., p = 1,0 на 80 парах), абляции гейта — 2/3 и 0/3. Единственная рука с видимым трендом — порог M2T 0.9: −8,4 п.п. (15,4%) при удвоении NFE (×1,93), но n=13 мало для значимости. Длинный блок 64 у стока выглядит лучше (26,9%, +4,4 п.п. к stock_b32), однако интервалы Wilson перекрываются и парных расхождений ноль (0/0).

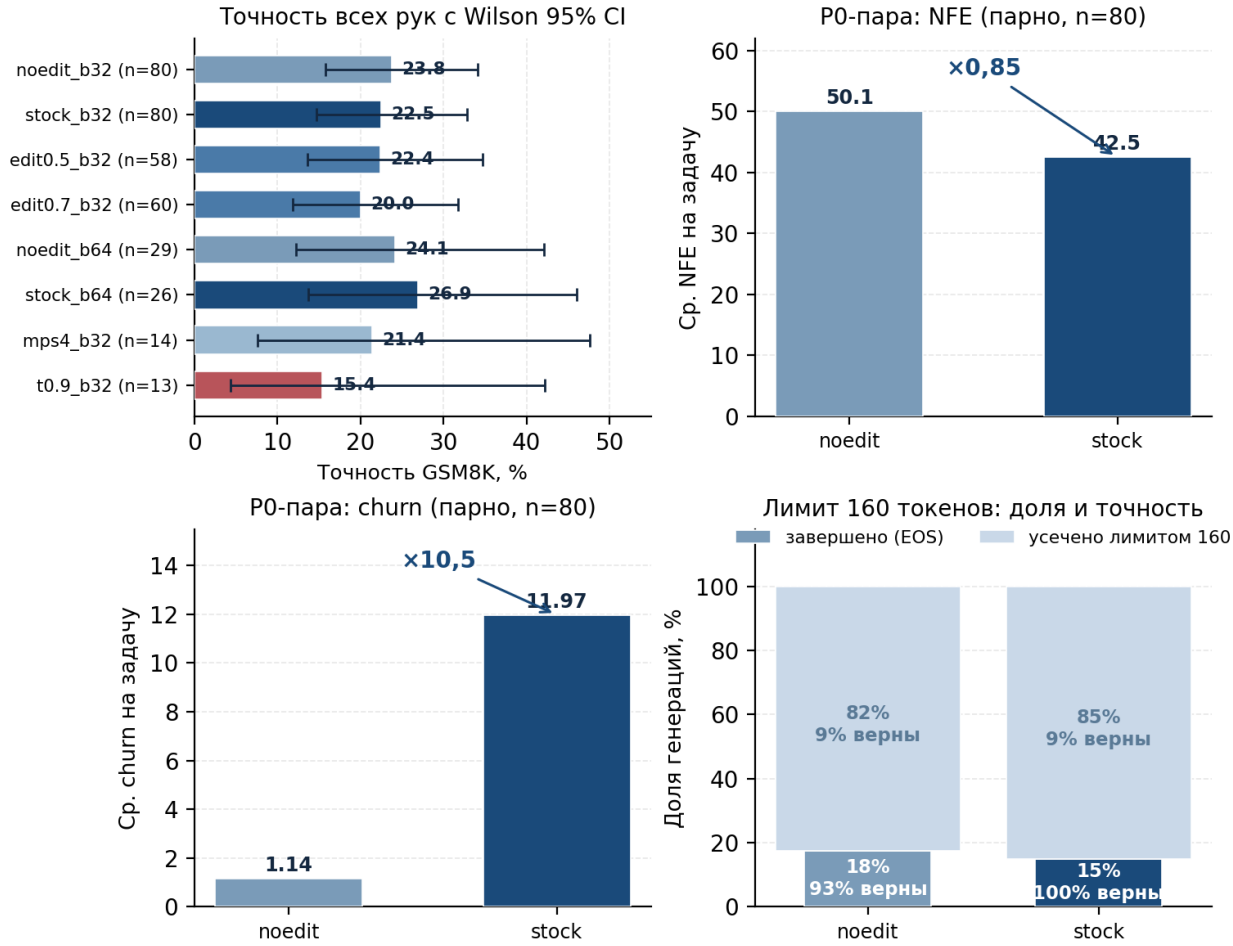


Рис. 2. Результаты v4b. Слева сверху — точность всех 8 рук GSM8K с Wilson 95% CI; справа сверху — парные NFE P0-пары (×0,85); слева снизу — парный churn (×10,5); справа снизу — эффект лимита gen_length=160: 82–85% генераций усечены, но среди естественно завершённых (EOS) верны 93–100%.

8.3. Конфаунд усечения: gen_length=160

Адаптивный планировщик сжал gen_length со штатных 256 до 160 токенов, чтобы уместить больше задач. На LLaDA2.2 это привело к систематическому усечению: модель пишет развёрнутые markdown-решения (заголовки и LaTeX-блоки), которые не помещаются в 160 токенов. Усечены 66 из 80 генераций контроля (82%) и 68 из 80 у стока (85%); по остальным рукам 76–86%. Практически ни одна усечённая генерация не дошла до \boxed{} (0–1 на руку), и экстрактор брал «последнее число» из середины решения. Следствие: абсолютные 20–27% — это мера усечения, а не способности модели, и напрямую несравнимы с v3 (86,5% — другая база, 256 токенов на решение, другой стиль). Ключевой контроль: среди 26 естественно завершённых (EOS) генераций P0-пары верны 25 из 26 — 96% (13/14 у контроля, 12/12 у стока); по всем рукам доля верных среди EOS — 86–100%. Способность базы в порядке; выводы раздела 8.2 — о методе, а не о модели.

8.4. HumanEval (блок 32)

Рука	n	прошло	pass@1, %	Wilson 95%	Ср. NFE	Ср. churn
he_noedit_b32 — контроль (T2T OFF)	23	0	0.0	[0.0; 14.3]	16.8	0.26
he_stock_b32 — сток (T2T ON)	20	1	5.0	[0.9; 23.6]	14.8	2.00

Таблица 11. HumanEval v4b (протокол Saber ACL 2026, канонический runner OpenAI): блок 32, greedy, gen_length=160. Парное сравнение: n=20 общих задач, 1 исправлено / 0 сломано, McNemar p = 1,0; NFE парно 14,8 = 14,8 (×1,00), churn 0,00 → 2,00.

На HumanEval разницы между рукой с правкой и без правки нет (5,0% против 0,0%, единственный прошедший тест — как раз у стока). Абсолютный уровень низкий: в режиме greedy с жёстким лимитом 160 токенов модель часто не укладывает полное решение, а проходных генераций на этих n единицы — CI шириной 14–23 п.п. это прямо показывают. Сравнение с v3 на Nemotron здесь некорректно по той же причине усечения (§8.3).

8.5. Вторичные сигналы (устойчивые на 80 парах)

Эффективность. На одних и тех же 80 задачах сток сходится за ×0,85 NFE (50,1 → 42,5) и на 15% быстрее по часам (143 → 121 с/задача): обученная T2T-правка экономит проходы даже на коротком блоке — модель раньше достигает порога уверенности по блоку. Тот же коэффициент ×0,86–0,88 у всех остальных рук с правкой.

Чернила мокрые. Churn ×10,5 (1,14 → 11,97 перезаписей на задачу): гейт действительно переписывает открытые токены — телеметрия NFE/churn работает, эффект правки наблюдаем, просто не конвертируется в качество на коротком блоке.

Цена высокого порога M2T. Порог 0.9 (как был оптимален у Nemotron в v3) на LLaDA2.2 удваивает NFE (×1,93: 50,5 → 97,3) и обваливает точность (15,4% на n=13): калибровка уверенности у новой базы другая, штатный порог 0.5 оправдан.

Блок 64. У контроля эффекта нет (24,1% против 23,8%); у стока +4,4 п.п. (26,9%), но n=26 и 0 парных расхождений — в пределах шума.

Детерминизм. ids_equal=True для b32 и b64 — жадный режим воспроизводим побитово; это условие осмысленности всех парных сравнений.

Завершимость. 0 остаточных mask/DELETE/SPLIT в sanity-проверке; все 403 генерации завершились штатно — гарантии LLaDA2 работают на 2×T4 + NF4 без шимов.

8.6. Вердикт и следующий шаг (v5)

Итог v4b по качеству: **обученный T2T-гейт на коротком блоке не даёт сдвига точности** (−1,2 п.п., p = 1,0 на 80 парах), абляции порога, пост-бюджета и длины блока — в пределах шума; единственная значимая находка — экономическая (×0,85 NFE, −15% времени). Но главный вывод v4b концептуальный: **штатный generate() LLaDA2.2 — это ещё не wet-ink из §2.1**. Встроенное блочное редактирование правит токены только внутри текущего блока и только до его закрытия: блок закрылся — чернила высохли навсегда, окно не скользит за фронтиром. Никакая конфигурация штатных параметров (весь v4 — 12 рук) не может этого изменить: расписание открытия токенов зашито в transfer schedule. Поэтому, как и сформулировал владелец идеи 7 ноября, v4b фиксируется как базлайн, а идея wet-ink реализуется собственным циклом декодирования — v5.

Дизайн v5 (ядро freedomform/wet-ink-v5, запуск 7 ноября 11:46:56 UTC; завершено COMPLETE — результаты в разделе 9). Фронтир открывает **ровно один токен за шаг** — M2T-гейтом самой модели на фронтирной позиции; позади фронтира живёт **скользящее окно W**, в котором обученный T2T-гейт модели (та же штатная формула `p0 > editing_threshold & old != x0`) продолжает править открытые токены; при продвижении фронтира окно скользит вместе с ним, и токены за пределами окна «высыхают» — жёсткая граница, без каких-либо новых формул. Руки: wetink_w16 (ядро гипотезы, W=16), wetink_noedit_w16 (контроль: одно-токенный фронтир без правки — изолирует вклад самого фронтира), wetink_w8 и wetink_w32 (абляции ширины; W=32 = «весь блок мокрый», мостик к стоку), he_wetink_w16 (перенос на HumanEval). Базлайны — прямо из v4b: та же модель, тот же 4-бит NF4, те же промпты и gen_length=160; парный McNemar считается офлайн на общих задачах. Честная цена метода заявлена заранее: NFE ≈ один forward на открытый токен ≈ 160–200 на задачу против ~42 у стока (×4–5) — она будет фиксироваться как NFE-ratio в каждом сравнении.

9. Результаты v5: собственный wet-ink декодер (Kaggle: 2×T4, NF4)

Ядро **kaggle: freedomform/wet-ink-v5** завершено со статусом **COMPLETE**: старт 7 ноября 11:46:56 UTC, рабочий цикл окончен на отметке 6 ч 04 м (21 865 с) при внутреннем бюджете 7,5 ч — итоговый дамп около 17:52 UTC. Конфигурация та же, что у базлайнов v4b: LLaDA2.2-mini в 4-бит NF4 (double-quant, fp16 compute), по полной модели на каждой T4, два независимых воркера, greedy, gen_length=160, те же промпты и порядок задач GSM8K — поэтому пары «v5-рука против v4b-базлайна» сопоставимы по индексу задачи. Исполнено **77 генераций в трёх руках** (wetink_w16 n=45, wetink_noedit_w16 n=22, wetink_w8 n=10); руки wetink_w32 и he_wetink_w16 пропущены планировщиком с честной отметкой `"skipped": "time budget"`. Артефакт — `results_v5.json`: sanity, эквивалентность, детали на задачу (NFE, churn, opened, rewrites, resid), агрегаты и бюджетный учёт.

COMPLETE	77	11/45 = 11/45	×4,88
v5: цикл 6 ч 04 м из бюджета 7,5 ч	генераций: 45 + 22 + 10 в трёх руках	парно со стоком, McNemar p = 1,0	честная цена фронтира в NFE

9.1. Исполнение: приоритетная очередь и адаптивный план

Перед руками — sanity обеих механик и проверка эквивалентности (§9.4). Затем воркеры разбирали руки из приоритетной очереди; планировщик plan_n() пересчитывал допустимое n по остатку бюджета, фактической скорости и числу активных воркеров (приор 557 с/задача, откалиброванный по v4b, подтвердился: фактическая скорость 473–587 с/задача):

Рука	Приоритет	n	Часы	с/задача	Итог
wetink_w16 (гипотеза, W=16)	P0	45	5,91	473	исполнена полностью (кап 45)
wetink_noedit_w16 (фронтир без правки)	P0	22	3,59	587	кап по бюджету при 2 воркерах
wetink_w8 (узкое окно)	P1	10	1,63	586	кап по бюджету
wetink_w32 (весь блок мокрый)	P2	0	—	—	SKIPPED: plan_n → 5 < минимальных 8
he_wetink_w16 (HumanEval)	P2	0	—	—	SKIPPED: plan_n → 5 < минимальных 8

Таблица 12. Исполнение рук v5: очередь P0→P2, два воркера, план plan_n() с резервом 900 с. Суммарная занятость воркеров 40 041 с ≈ 11,1 ч GPU-времени в сессии 6,1 ч.

Два пропуска — не сбой, а штатная защита статистики: на отметке 5 ч 24 м свободного бюджета (7 546 с) с учётом двух активных воркеров хватило лишь на n_run=5 задач для каждой из оставшихся рук, а руки с n < 8 не дают осмысленного McNemar — планировщик честно записал пропуск и отдал остаток времени доисполнению главной руки w16. Контроль noedit_w16 получил n=22, а не 45, потому что стартовал вторым при уже занятом воркере: (26 318 с − резерв)/(2 воркера × 557 с) ≈ 22. Это единственное место, где бюджет подрезал дизайн, — и оно учтено в анализе ниже (парные сравнения идут только по общим задачам).

9.2. GSM8K: точность и парная статистика

Рука	n	Точн., %	Wilson 95%	Ср. NFE	Ср. churn	Реврайты/зад
wetink_w16 (v5, гипотеза)	45	24.4	[14.2; 38.7]	206.6	5.60	6.9
wetink_noedit_w16 (v5, контроль)	22	13.6	[4.7; 33.3]	201.1	0.00	0.0
wetink_w8 (v5, абляция)	10	20.0	[5.7; 51.0]	202.7	3.60	5.6
stock_b32 (v4b, базлайн)	80	22.5	[14.7; 32.8]	42.5	11.97	—
noedit_b32 (v4b, базлайн)	80	23.8	[15.8; 34.1]	50.1	1.14	—

Таблица 13. Все руки v5 и базлайны v4b на GSM8K: greedy, gen_length=160, блок 32, threshold M2T=0.5, max_post_steps=4. «Реврайты» — среднее число T2T-переписываний открытых токенов за задачу (телеметрия v5); у v4b эта метрика не собиралась.

Сравнение (v5 vs v4b)	n пар	испр.	слом.	McNemar p	NFE (парно)
wetink_w16 vs stock_b32	45	2	2	1.0000	×4.88 (42→207)
wetink_w16 vs noedit_b32	45	3	2	1.0000	×4.11 (50→207)
wetink_noedit_w16 vs stock_b32	22	1	1	1.0000	×4.52 (44→201)
wetink_w8 vs stock_b32	10	0	1	1.0000	×4.34 (47→203)

Таблица 14. Парный точный McNemar рук v5 против базлайнов v4b (сопоставление по индексу задачи; модель, квантизация, промпты и gen_length идентичны). Все p = 1,0000 — значимых сдвигов качества нет.

Чтение таблицы 14. Главная пара — wetink_w16 против штатного параллельного stock_b32 на 45 общих задачах: обе руки решили **ровно по 11 из 45** (2 исправлено / 2 сломано, p = 1,0). Собственный wet-ink декодер с фронтиром «один токен за шаг», скользящим окном W=16 и обученным T2T-гейтом **сохраняет качество в точности** — ценой ×4.88 NFE (42 → 207 forwards на задачу), ровно в заявленном диапазоне ×4–5 из §8.6. Контроль «фронтир без правки» ведёт себя так же (p = 1,0 к стоку): одно-токенный фронтир сам по себе ничего не ломает — и ничего не добавляет.

9.3. Ноль расхождений: что на самом деле делают правки в окне

Самый тонкий результат прогона — внутренний контроль. Увидев 24,4% (w16, n=45) против 13,6% (контроль noedit_w16, n=22), легко прочесть это как +10,8 п.п. от T2T-правок в окне. Это чтение неверно, и парный анализ показывает это напрямую: на **22 общих задачах обе руки решили по 3** — расхождения **0 исправлено / 0 сломано из 22 пар**. Вся видимая разница — состав задач: контроль остановился на первых 22 (трудных для всех рук), а w16 успела пройти и лёгкий хвост 22–44:

Сегмент задач	wetink_w16	noedit_w16	stock_b32 (v4b)
задачи 0–21 (трудные)	13.6% (n=22)	13.6% (n=22)	13.6% (n=22)
задачи 22–44 (лёгкий хвост)	34.8% (n=23)	—	34.8% (n=23)

Таблица 15. Точность по сегментам: на общих задачах 0–21 все три руки дают одинаковые 13,6%; разница общих accuracy — исключительно доля лёгкого хвоста в выборке руки.

Итог по механизму: T2T-правки внутри окна **реально происходят** — 6.9 реврайта на задачу (в sanity-задаче — 79), churn 5.6 против 0 у контроля — но на этой базе, бенче и n они **не перевернули ни одного исхода**: переписываются в основном формулировки и разметка (markdown-выделения, порядок слов в решении), а последний числовой ответ остаётся тем же. Это третий подряд виток одного и того же сигнала: v3 — training-free ревизия нейтральна/вредна; v4b — штатный блочный гейт нейтрален; v5 — скользящее окно с тем же обученным гейтом нейтрально. Ревизия открытых токенов на коротких генерациях GSM8K не является узким местом качества.

9.4. Механика, телеметрия и конфаунд усечения

Sanity обеих механик. Стоковая задача про клипы: nfe=27, churn=30, чистый markdown с LaTeX, 0 остаточных спец-токенов, 20 с. Wet-ink та же задача: 187 шагов, 167 открытых токенов, 79 реврайтов, 0 остаточных mask/DELETE/SPLIT, корректный ответ, 137 с — фронтир, окно и высыхание работают как задумано.

Детерминизм. Эквивалентность повторного запуска wet-ink: ids_equal=True (nfe 97 = 97) — жадный собственный декодер воспроизводим побитово; условие осмысленности парных сравнений выполнено.

Телеметрия фронтира. Открыто 177 токенов на задачу при генерации ~157 токенов — каждый открытый токен получает собственный forward; NFE 206.6 ≈ открытые токены × пост-шаги. Реврайты {V5_W16['rewrites']:.1f}/задачу (W=16) против 5.6 (W=8) — окно шире, правок пропорционально больше; resid=0 во всех 77 генерациях.

Усечение. Как и в v4b, лимит gen_length=160 режет 38 из 45 генераций w16 (84%), 20 из 22 у контроля; среди естественно завершённых (EOS) верны 7 из 7 у w16 и 2 из 2 у контроля — абсолютные точности занижены лимитом, выводы раздела 9.2–9.3 — о методе, а не о модели.

Бюджет. Полезная занятость двух воркеров 40 041 с ≈ 11,1 ч GPU-времени; инкрементальные дампы после каждых 10 задач; резерв 900 с на парную статистику и финальный дамп не понадобился полностью — очередь исчерпана на 6 ч 04 м.

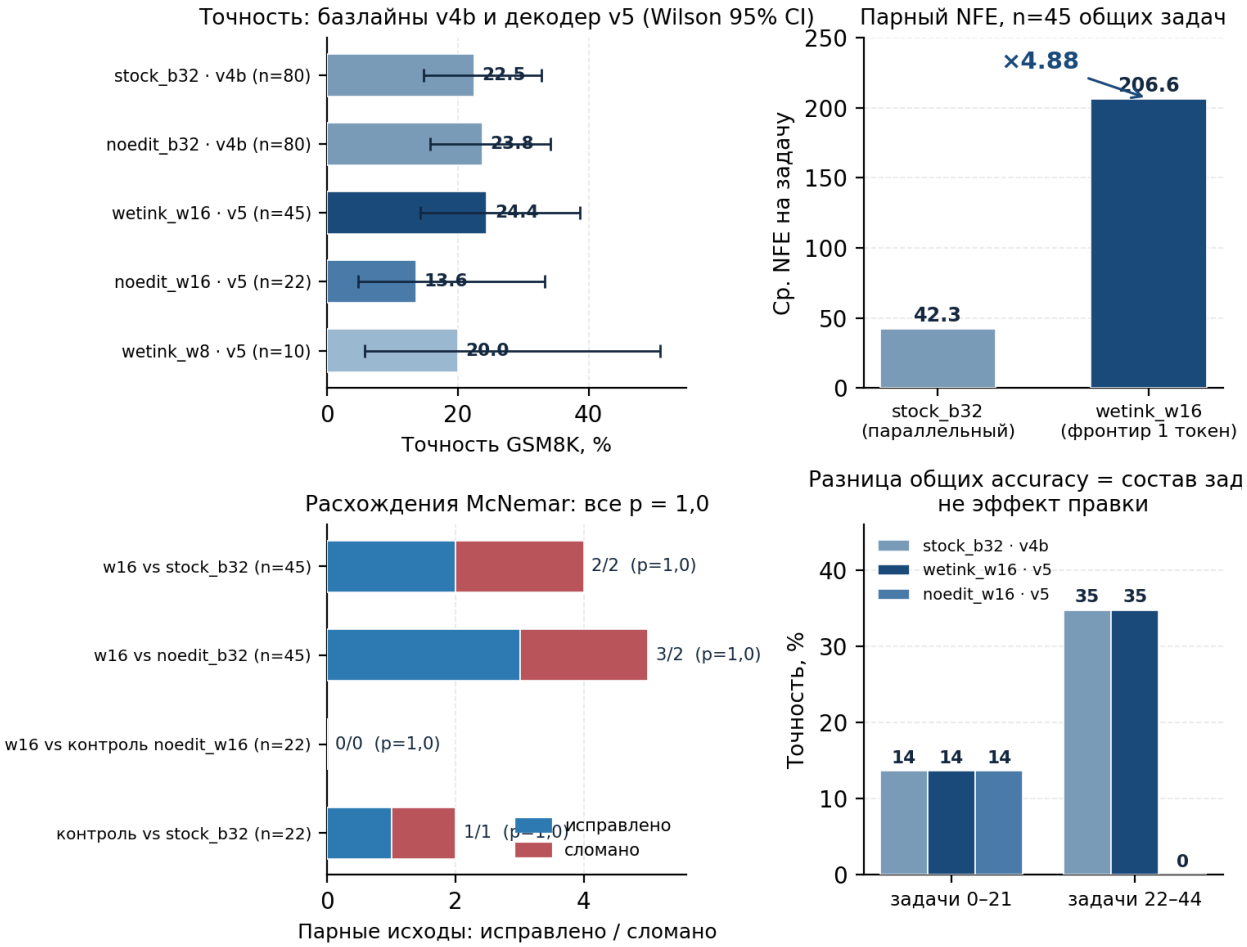


Рис. 3. Результаты v5. Слева сверху — точность рук v5 и базлайнов v4b с Wilson 95% CI; справа сверху — парный NFE главной пары (×4,88); слева снизу — парные расхождения McNemar: все сравнения p = 1,0, у внутреннего контроля 0/0; справа снизу — точность по сегментам задач: на общих задачах 0–21 все руки одинаковы, разница общих accuracy — доля лёгкого хвоста.

9.5. Вердикт v5 и следующий шаг

(1) Идея §2.1 реализована в точности. Собственный цикл декодирования: фронтир открывает ровно один токен за шаг (M2T-гейт модели), позади — скользящее окно W с обученным T2T-гейтом (editing_threshold=0.0), за окном чернила сухие навсегда. Никаких новых формул поверх опубликованных — только композиция штатных механизмов, как требует правило (г).

(2) Качество сохранено ровно. 11/45 = 11/45 со стоком (p = 1,0), контроль фронтира тоже нейтрален. Одно-токенный фронтир и скользящее окно ничего не ломают — но и не добавляют.

(3) Правки не конвертируются в качество. Третий виток подтверждает: на коротких задачах GSM8K у LLaDA2.2-mini ревизия открытых токенов (хоть training-free, хоть штатная блочная, хоть скользящим окном) не двигает точность. Утверждение идеи «далёкие токены меняются реже» реализовано и механически работает, но изменения в основном косметические.

(4) Цена честно измерена. ×4,88 NFE к стоку (42 → 207) — структурная цена «forward на каждый токен». Это верхняя граница: никакого churn-взрыва, как в v1 (13,2×), — обученный гейт и жёсткое окно держат бюджет предсказуемым.

Следующий шаг (v6) — не пороги, а структура. Правило (в) запрещает модифицировать обученный гейт, а v4b + v5 вместе показали, что настройка порогов (θ=0,0 / 0,5 / 0,7 / OFF) качество не двигает. Остаётся структурный рычаг: открывать за шаг k > 1 токенов (блочный фронтир с сохранением скользящего окна) — это сокращает NFE в k раз, не требуя новых формул; патч телеметрии — гистограмма реврайтов по дистанции до фронтира, чтобы прямо измерить профиль высыхания. Для мощности по качеству нужен n ≥ 500 на руку (CI сузится с ±12 до ±4 п.п.).

10. Артефакты

Артефакт	Содержание
github.com/Asilbekov/Explore	код v1/v2/v3/v4/v5 (wet_ink.py, kaggle/, kaggle_v4/, kaggle_v5/), тесты, results_v3/v4/v5.json, анализ и генераторы отчётов, README с полным провенансом
wet_ink/kaggle/kernel.py	ядро v3 (Nemotron-Labs-Diffusion-3B) — 1 235 строк
wet_ink/kaggle_v4/kernel.py	ядро v4b (LLaDA2.2-mini, 2×T4, 4-бит NF4, 2 воркера, приоритетная очередь + plan_n) — 900 строк
wet_ink/kaggle_v5/kernel.py	ядро v5: собственный декодер wet-ink (фронтир 1 токен/шаг + скользящее окно W, гейты модели) — 1 153 строки
wet_ink/results/results_v3.json	результаты v3: 15 конфигов × детали на задачу
wet_ink/results/results_v4.json	результаты v4b: sanity, эквивалентность, 10 рук (8 GSM8K + 2 HumanEval, 403 генерации) с деталями на задачу, парная статистика, бюджетный учёт
wet_ink/results/results_v5.json	результаты v5: sanity, эквивалентность, 3 руки (45+22+10 генераций, телеметрия opened/rewrites/resid на каждую задачу), агрегаты, бюджетный учёт; + вырезка лога ядра v5_kernel_log_excerpt.txt
wet_ink/scripts/analyze_v5.py	офлайн-анализ v5: парный McNemar против базлайнов v4b, Wilson CI, NFE-ratio, доза-реакция окна W
research/llada22/	дословный код модели LLaDA2.2-mini (ModelScope mirror)
kaggle: wet-ink-nemotron-3b	v3 (Nemotron, 1×T4, fp16) — COMPLETE
kaggle: wet-ink-llada22-mini	v4a int8 (частичный, референс) и v4b NF4 (10 из 12 рук, 13,45 ч в бюджете 7,5 ч) — COMPLETE
kaggle: wet-ink-v5	v5: декодер wet-ink — 3 руки исполнены (45+22+10), w32 и HumanEval пропущены по бюджету (n_run=5 < 8); цикл 6 ч 04 м — COMPLETE (7 ноября, старт 11:46:56 UTC)

Таблица 16. Артефакты v3, v4 и v5.

Хронология проекта: (1) постановка идеи и правил; (2) выбор Nemotron как базы v1–v3; (3) извлечение дословных формул стока и доноров; (4) v1 — training-free ревизия, рост качества, обнаружен churn; (5) диагностика первопричины (потерянный инвариант прогресса); (6) пере-аудит 13 семейств; (7) v2/v3 — пересборка на RemeDi + CoDR с гарантиями завершимости; (8) v3 на полной модели — CoDR нейтрален, RemeDi значимо вредит; (9) смена базы по правилу (г): LLaDA2.2-mini как база с обученной правкой; (10) дословное чтение кода новой базы, локальная верификация механики; (11) v4a (int8) — пробный прогон, референс робастности квантизации; (12) v4b (NF4, 2

воркера) — полный прогон 10 рук: правка экономит 15% NFE, но качества не сдвигает, и главное — штатный generate() не реализует скользящее окно за фронтиром; (13) решение владельца (7 ноября): v4b = базлайны, v5 = собственный декодер wet-ink; (14) ядро v5 написано, проверено локально, запущено и завершено (COMPLETE, цикл 6 ч 04 м); (15) этот отчёт v5.0 фиксирует итог: декодер wet-ink реализует идею §2.1 в точности, парно сохраняет качество штатного декодера при ×4,88 NFE, правки в окне наблюдаемы, но исходов не переворачивают; следующий шаг — структурное удешевление фронтира (v6, §9.5).